

Spécification — Déclenchement proactif SLO

QoS Orchestrator — PiCar-X

Document de référence pour le dimensionnement des variables de démo (θ , v , T_{send} , B , A , D_{MIN} , D_{MAX}) du système QoS Orchestrator — PiCar-X. Couvre le modèle physique, les formules, les règles de conception, trois scénarios numériques validés par logs réels, la cohérence avec l'implémentation (hystérésis, TOPSIS, mode enhanced), et la procédure à suivre pour changer une variable sans casser la stabilité du système.

Contents

Résumé exécutif — configuration recommandée	2
1 Contexte et objectif	3
2 Modèle de latence basée sur la distance	3
2.1 Principe	3
2.2 Formule	3
2.3 Représentation graphique	3
3 Définitions des métriques et objectifs	4
3.1 Distance de violation — D_{slo}	4
3.2 Distance proactive — D_{proac}	4
3.3 Marge spatiale — ΔD	4
3.4 Déplacement par cycle — δ	4
3.5 Temps disponible pour réagir — T_{proac}	5
3.6 Condition minimale de proactivité	5
3.7 Rôle des prédictions ML	5
4 Variables du système	5
5 Formules fondamentales	6
6 Règles de conception (contraintes)	6
7 Graphe de dépendance	6
8 Démarche de calcul	7
9 Application numérique — trois scénarios testés	7
9.1 Paramètres communs (mesurés/confirmés)	7
9.2 Scénario A — $\theta=60$ ms, $v=0,5$ cm/s (<i>cas confirmé stable</i>)	8
9.3 Scénario B — $\theta=60$ ms, $v=1,0$ cm/s (<i>échec confirmé — mêmes T_{send}/T_{cycle}</i>)	8
9.4 Scénario C — $\theta=100$ ms, $v=1,0$ cm/s (<i>compensation par le seuil, cas confirmé</i>)	8
9.5 Simulation détaillée du déclenchement — Scénario A (point $d = D_{proac} = 11,21$ cm)	9

10 Tableau comparatif final	9
11 Cohérence avec l'implémentation et limites	9
11.1 Traçabilité des valeurs dans le code	9
11.2 Mécanisme de quantification $T_{send} \rightarrow T_{cycle}$	10
11.3 Les prédictions ne sont pas géométriques	10
11.4 Mode réactif = filet de sécurité	10
11.5 Optimisation Collector — cache de fond, gain non confirmé en pratique	10
11.6 Normalisation TOPSIS et marge d'hystérésis (5 %)	11
11.7 Mode enhanced — SLOs CPU/RAM en ressource absolue (cœurs/Go)	11
12 Justification synthétique du choix	11
13 Procédure de changement de variable — garder le système stable	12
13.1 Carte d'impact (à consulter en premier)	12
13.2 Procédure A — Changer θ (seuil SLO)	12
13.3 Procédure B — Changer v (vitesse voiture)	13
13.4 Procédure C — Changer T_{send} (donc T_{cycle} , via quantification)	13
13.5 Procédure D — Changer B, A, D_{MIN}, D_{MAX} (formule matérielle — rare)	13
13.6 Ce qui ne nécessite jamais de recalcul géométrique	14

Résumé exécutif — configuration recommandée

	Configuration recommandée	Alternative — démo plus dynamique
θ (seuil)	100 ms	100 ms
v (vitesse)	0,5 cm/s	1,0 cm/s
T_{send}	5 s	5 s
Statut	✓ confirmé par logs réels — $D_{proac}=32,45$ cm, migration proactive cycle #103 (cloud2→edge1), 17+ migrations propres observées sur un run étendu, 0 réactif	✓ confirmé par logs réels (cycles 102-108) — migration proactive cloud2→edge1, marge $D_{proac}=11,45$ cm (plus fine, mais toujours positive)
Usage	scénario par défaut, marge maximale	voiture plus rapide, marge plus fine mais toujours sûre

Pourquoi $\theta=100$ ms et pas 60 ms ? À $\theta=60$ ms, la marge de proactivité (D_{proac}) ne reste positive qu'à $v=0,5$ cm/s ; dès que $v=1,0$ cm/s, la fenêtre d'anticipation dépasse la distance disponible et le système reste en alerte permanente (**pred_breach=True** en continu, détail §9.3). $\theta=100$ ms restaure une marge confortable aux deux vitesses testées (§§9.2-9.4), ce qui en fait le réglage le plus robuste pour la démo.

Trois éléments complètent ce dimensionnement géométrique et sont détaillés en §§11.5-11.7 :

1. Une marge d'hystérésis anti-ping-pong de **5 %** (**decision.py**) combinée à un garde-fou de normalisation TOPSIS (§11.6) stabilisent la *décision* de migration, indépendamment de la géométrie de *déclenchement* dimensionnée ici.
2. Le mode enhanced (intentions utilisateur via LLM) exprime les besoins CPU/RAM en **ressource absolue** (cœurs/Go) plutôt qu'en % de charge, pour tenir compte de l'hétérogénéité du parc de VMs (§11.7).
3. L'optimisation du Collector (cache de fond) est décrite en §11.5, avec ses limites actuelles.

1 Contexte et objectif

Le système de QoS orchestre la migration d'un service entre plusieurs VMs OpenStack en fonction de la latence mesurée entre une voiture simulée (PiCar-X) et les VMs. L'objectif est de migrer le service **avant** que la latence dépasse le seuil SLO contractuel θ , en exploitant les prédictions fournies par les APIs de machine learning (k-NN, horizon 7).

Le système décide selon deux régimes :

- **Proactif (régime nominal)** : les prédictions ML anticipent une violation future — la migration est déclenchée avant que la valeur réelle atteigne θ .
- **Réactif (filet de sécurité uniquement)** : utilisé seulement lorsque les prédictions ML sont indisponibles (API en panne, historique insuffisant). La décision se rabat alors sur la valeur mesurée courante.

Implémentation (`services/decision_intelligence/violation_detector.py`) : la décision est pilotée par la prédiction pour **toutes** les métriques (primaire + secondaires). Le réactif n'est qu'un repli sans prédiction (§11.4). De plus :

- une **Règle A** (`gate, decision.py`) restreint le déclenchement de migration à une violation de la métrique **primaire** (latence) — une violation secondaire seule (cpu/ram) ne déclenche jamais de migration ;
- une **hystérésis de 5 %** (`_MIGRATION_MARGIN, decision.py`, voir §11.6) empêche de migrer vers un candidat qui ne dépasse pas nettement le score de la VM active.

Ces deux mécanismes sont **complémentaires** à la présente spécification géométrique : ils stabilisent la *décision*, cette spec dimensionne le *déclenchement*.

2 Modèle de latence basée sur la distance

2.1 Principe

La latence simulée entre la voiture et une VM dépend de la distance euclidienne entre leurs positions sur la carte. Plus la voiture s'éloigne, plus la latence croît **linéairement** entre une valeur minimale B (voiture très proche) et une valeur maximale A (voiture à distance maximale).

2.2 Formule

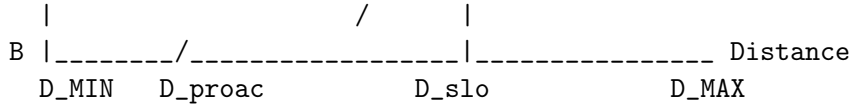
$$L(d) = \frac{d - D_{MIN}}{D_{MAX} - D_{MIN}} \times (A - B) + B \quad (1)$$

Si distance =	Latence =
D_{MIN}	B (minimum)
D_{MAX}	A (maximum)
entre les deux	interpolation linéaire

2.3 Représentation graphique

Latence





3 Définitions des métriques et objectifs

3.1 Distance de violation — D_{slo}

Définition : distance (cm) voiture $\leftrightarrow$ VM à partir de laquelle la latence simulée atteint exactement le seuil θ . **But :** outil de **conception**. Le système ne connaît pas les distances — il mesure des latences en ms. D_{slo} traduit θ (ms) en cm pour raisonner sur la carte : « à partir de combien de cm la migration doit-elle être déjà faite ? »

$$D_{slo} = D_{MIN} + \frac{\theta - B}{\text{slope}} \quad (2)$$

D_{slo} n'est **pas** une variable du code — c'est la traduction en cm du seuil θ .

3.2 Distance proactive — D_{proac}

Définition : distance à partir de laquelle les prédictions ML annoncent une violation future, donc le point de la carte où la migration proactive se déclenche. **But :** savoir *où* sur la carte le système réagit. Si le proactif fonctionne, la migration a lieu à D_{proac} et le service est déjà migré quand la voiture atteint D_{slo} .

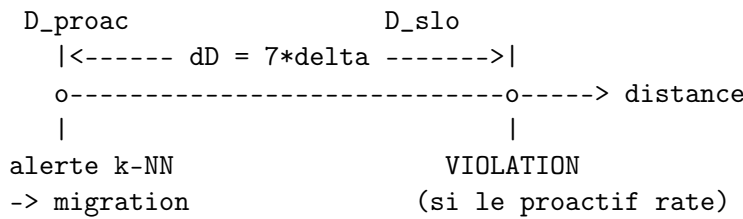
$$D_{proac} = D_{slo} - 7 \times \delta \quad (3)$$

Le terme $7 \times \delta$ représente les 7 cycles de prédiction disponibles (horizon = 7).

3.3 Marge spatiale — ΔD

Définition : espace (cm) entre le point de déclenchement proactif et le point de violation. **But :** « le système a-t-il assez d'espace (donc de temps) pour détecter *et* exécuter la migration avant la violation ? »

$$\Delta D = D_{slo} - D_{proac} = 7 \times \delta \quad (4)$$



3.4 Déplacement par cycle — δ

Définition : distance parcourue par la voiture entre deux cycles de décision consécutifs.

$$\delta = v \times T_{cycle} \quad [\text{cm/cycle}] \quad (5)$$

T_{cycle} = période **effective** d'un cycle d'orchestration — pas T_send (intervalle d'envoi picar réglé). Ce n'est exact que si $T_{cycle} = T_{send}$, ce qui n'est presque jamais le cas (cf. §11.2, mécanisme de quantification).

3.5 Temps disponible pour réagir — T_{proac}

Définition : temps entre l’alerte proactive et le moment où la violation réactive surviendrait.

$$T_{proac} = \frac{\Delta D}{v} = 7 \times T_{cycle} \quad (6)$$

T_{proac}	Interprétation
$< 2 \cdot T_{cycle}$	trop court — migration probablement tardive
$= 2 \cdot T_{cycle}$	minimum acceptable
$= 7 \cdot T_{cycle}$	maximum (horizon = 7 prédictions)

Pourquoi 2 cycles minimum ?

Cycle n : k-NN detecte la violation future -> decision MIGRATE envoyee

Cycle n+1 : migration kubect1 executee -> nouvelle VM active

Total = 2 x T_cycle secondes necessaires

3.6 Condition minimale de proactivité

$$T_{proac} \geq 2 \times T_{cycle} \iff \Delta D \geq 2 \times \delta \quad (7)$$

Comme $\Delta D_{max} = 7\delta$, la condition est toujours vérifiée tant que la voiture **ne traverse pas D_{slo} en un seul cycle** :

$$v \times T_{cycle} < D_{slo} - D_{MIN} \quad (8)$$

Ce n’est pas θ qui contraint la proactivité (il n’y a pas de facteur α) — c’est directement v et T_{cycle} .

3.7 Rôle des prédictions ML

Les APIs (5001 latence, 5002 CPU, 5003 RAM) retournent 7 valeurs futures par cycle :

API recoit : historique des 50 dernieres valeurs

API retourne: [pred[0], pred[1], ..., pred[6]]

cycle suivant

7e cycle futur

pred[k] = valeur estimée dans $(k+1) \times T_{cycle}$ secondes. Déclenchement proactif si **au moins une** prédiction dépasse θ :

```
pred_breach = any(pred[k] > theta for k in 0..6) # comparaison directe a theta, sans
facteur alpha
```

4 Variables du système

Variable	Description	Statut
θ	Seuil SLO (ms)	config
v	Vitesse voiture (cm/s)	dashboard
T_{send}	Intervalle d’envoi picar réglé (s)	config picar

Variable	Description	Statut
T_{cycle}	Période effective d'un cycle — celle qui compte dans les formules	mesurée, pas réglée directement
horizon	7 prédictions ML	fixe (API)
B	Latence minimale (ms) — voiture très proche	scripts VM
A	Latence maximale (ms) — voiture très loin	scripts VM
D_{MIN}	Distance minimale (cm)	scripts VM
D_{MAX}	Distance maximale (cm)	scripts VM

5 Formules fondamentales

$$L(d) = \frac{d - D_{MIN}}{D_{MAX} - D_{MIN}}(A - B) + B \quad \text{slope} = \frac{A - B}{D_{MAX} - D_{MIN}} \text{ [ms/cm]} \quad (9)$$

$$D_{slo} = D_{MIN} + \frac{\theta - B}{\text{slope}} \quad \delta = v \times T_{cycle} \quad (10)$$

$$\text{pred}[k] \approx L(d + (k + 1)\delta), \quad k = 0..6 \quad \exists k : \text{pred}[k] > \theta \iff d > D_{slo} - (k + 1)\delta \quad (11)$$

$$\Delta D_{max} = 7\delta \quad \Delta D_{min} = \delta \quad T_{proac}^{max} = 7 T_{cycle} \quad D_{proac} = D_{slo} - 7\delta \quad (12)$$

Mécanisme de quantification $T_{send} \rightarrow T_{cycle}$ (*découverte empirique, critique*) :

$$T_{cycle} = \max \left(\lceil T_{send}/2 \rceil \times 2, T_{traitement} \right) \quad (13)$$

Le picar tique à intervalle **fixe de 2 s** (`setInterval(tickBridge, 2000)` dans `picarx_sim.html`) ; un envoi au hub ne part que si maintenant – dernier _envoi $\geq T_{send}$. Donc l'intervalle réel entre deux envois réussis est le **prochain multiple de 2 s** $\geq T_{send}$ — et si ce multiple est trop juste par rapport au temps de traitement réel du cycle ($T_{traitement}$), l'envoi rate son tick et saute au suivant, causant du **jitter**.

6 Règles de conception (contraintes)

#	Règle	Justification
1	$B < \theta/3$	zone « saine » suffisamment large
2	$A > 2\theta$	zone de violation assez large sur la carte
3	$v < (D_{slo} - D_{MIN})/T_{cycle}$	pas de traversée de D_{slo} en un seul cycle
4	$D_{MAX} > D_{slo}$	toujours vrai si $\theta < A$
5	$D_{proac} > D_{MIN}$	déclenchement dans la plage physique
6	$\lceil T_{send}/2 \rceil \times 2 > T_{traitement} + \text{marge}$	cadence stable, pas de jitter (§11.2)
—	$B < \theta < A$	contrainte absolue
—	$\theta > B + 15 \text{ ms}$	zone saine visible en démo

7 Graphe de dépendance

T_send, T_traitement

|

v

T_cycle = ceil(T_send/2)*2 (borne en dessous par T_traitement)

```

|
v
theta, v, T_cycle (parametres d'entree du calcul geometrique)
|
v
delta = v x T_cycle
|
v
dD_max = 7 x delta
|
B, A, D_MIN, D_MAX (choix de conception / materiel)
|
+-----+
|
v
slope = (A - B)/(D_MAX - D_MIN)
v
D_slo = D_MIN + (theta - B)/slope
v
D_proac = D_slo - 7*delta
v
v_max = (D_slo - D_MIN)/T_cycle
v
Verification des 6 regles

```

8 Démarche de calcul

Étape	Action	Formule
0	Mesurer $T_{traitement}$ réel	logs hub, ≥ 10 cycles
1	Fixer T_{send} , en déduire T_{cycle}	$\lceil T_{send}/2 \rceil \times 2$, vérifier $> T_{traitement}$
2	Fixer θ, v	paramètres imposés
3	Calculer δ	$\delta = v \times T_{cycle}$
4	Calculer ΔD_{max}	7δ
5	Choisir B	$B < \theta/3$
6	Choisir A	$A > 2\theta$
7	Choisir D_{MIN}	distance physique mini réaliste
8	Choisir D_{MAX}	distance max réelle de la trajectoire
9	Calculer slope	$(A - B)/(D_{MAX} - D_{MIN})$
10	Calculer D_{slo}	$D_{MIN} + (\theta - B)/slope$
11	Calculer D_{proac}	$D_{slo} - 7\delta$
12	Vérifier v_{max}	$(D_{slo} - D_{MIN})/T_{cycle}$
13	Vérifier les 6 règles	tableau §6

9 Application numérique — trois scénarios testés

9.1 Paramètres communs (mesurés/confirmés)

$B=5$, $A=150$, $D_{MIN}=3$, $D_{MAX}=80$ -> slope = $145/77 = 1.883$ ms/cm
 $T_{send} = 5$ s -> $T_{cycle} = \text{ceil}(5/2) \times 2 = 6$ s (CONFIRME : cadence rock-solid a 6s dans les logs,
contre 4/6/8s jitter a $T_{send}=4s$)

9.2 Scénario A — $\theta=60$ ms, $v=0,5$ cm/s (*cas confirmé stable*)

```
delta = 0.5 x 6 = 3.0 cm/cycle
D_slo = 3 + (60-5)/1.883 = 32.21 cm
D_proac = 32.21 - 7x3.0 = 11.21 cm
v_max = (32.21-3)/6 = 4.87 cm/s
```

Règle	Calcul	Résultat
$B < \theta/3$	$5 < 20$	✓
$A > 2\theta$	$150 > 120$	✓
$v < v_{max}$	$0.5 < 4.87$	✓
$D_{MAX} > D_{slo}$	$80 > 32.21$	✓
$D_{proac} > D_{MIN}$	$11.21 > 3$	✓ marge confortable

Preuve empirique (logs, run $v=0,5$) : après une migration unique vers cloud1, le système reste STAY sur des **dizaines de cycles consécutifs** (86→98+), scores TOPSIS 0,90–0,97, **aucun ping-pong**, malgré des SLOs secondaires (cpu/ram) qui entrent et sortent du jeu de SLOs à chaque cycle.

9.3 Scénario B — $\theta=60$ ms, $v=1,0$ cm/s (*échec confirmé — mêmes T_{send}/T_{cycle}*)

```
delta = 1.0 x 6 = 6.0 cm/cycle
D_slo = 32.21 cm (identique -- independant de v)
D_proac = 32.21 - 7x6.0 = -9.79 cm
```

Règle	Calcul	Résultat
$D_{proac} > D_{MIN}$	$-9.79 < 3$	× VIOLÉE

Preuve empirique (logs, run $v=1,0$) : dès la migration vers cloud1, `pred_breach=True` sur **tous** les cycles suivants sans interruption (34→41+), latence mesurée qui stagne à 59,8 ms pendant 15+ cycles alors que le système reste en alerte permanente. La fenêtre d'anticipation (42 cm) dépasse toute la distance disponible (29,2 cm) : le système « voit » une violation future même quand la voiture vient d'arriver.

9.4 Scénario C — $\theta=100$ ms, $v=1,0$ cm/s (*compensation par le seuil, cas confirmé*)

```
D_slo = 3 + (100-5)/1.883 = 53.45 cm
D_proac = 53.45 - 42 = 11.45 cm <- quasi identique a la marge du Scenario A (11.21 cm)
v_max = (53.45-3)/6 = 8.41 cm/s
```

Règle	Calcul	Résultat
$D_{proac} > D_{MIN}$	$11.45 > 3$	✓ (marge équivalente au scénario A)

Preuve empirique (logs, cycles 102-108) : migration proactive cloud2 → edge1 observée, cohérente avec la marge calculée. Un run à $\theta=100/v=0,5$ (marge $D_{proac}=32,45$ cm, encore plus confortable) confirme la même stabilité sur un run étendu — 17+ migrations propres, 0 réactif (voir Résumé exécutif). $\theta=100$ ms est donc validé aux deux vitesses testées, et constitue la configuration recommandée pour la démo.

9.5 Simulation détaillée du déclenchement — Scénario A (point $d = D_{proac} = 11, 21$ cm)

k	distance prédite (cm)	L (ms)	$> \theta$ (60ms) ?
0	14.21	24.3	non
1	17.21	30.0	non
2	20.21	35.6	non
3	23.21	41.2	non
4	26.21	46.9	non
5	29.21	52.5	non
6	32.21	60.0	oui → PROACTIF

Le déclenchement intervient sur la **7^e prédiction**, avec 7 cycles d'avance (42 s à $T_{cycle}=6$ s).

10 Tableau comparatif final

	A	B	C
θ	60 ms	60 ms	100 ms
v	0,5 cm/s	1,0 cm/s	1,0 cm/s
T_{send}	5 s	5 s	5 s
T_{cycle}	6 s	6 s	6 s
D_{slo}	32.21 cm	32.21 cm	53.45 cm
D_{proac}	11,21 cm	−9,79 cm	11,45 cm
Règle 5	✓	×	✓
Statut	logs réels : stable	logs réels : alarme permanente	logs réels : migration proactive confirmée (cycles 102-108)

$\theta=100$ ms est la valeur configurée dans `shared/config.py` ; un run supplémentaire à $\theta=100/v=0,5$ ($D_{proac}=32,45$ cm, encore plus confortable que les 3 scénarios ci-dessus) confirme la même stabilité sur plusieurs dizaines de cycles — c'est la configuration recommandée pour la démo (voir Résumé exécutif).

11 Cohérence avec l'implémentation et limites

11.1 Traçabilité des valeurs dans le code

Valeur	Fichier
$B=5$, $A=150$, $D_{MIN}=3$, $D_{MAX}=80$ $v = 0.5 / 1.0$ cm/s	scripts VM *_ping_fixeCarac.py et infrastructure/picarx_sim.html (FML) slider vitesse picarx_sim.html / dashboard :8080
$\theta = 100$ ms	shared/config.py → METRICS_REGISTRY["latency"]["default_threshold"]
$T_{send} = 5$ s	infrastructure/picar_bridge.py → SEND_INTERVAL_S

Valeur	Fichier
horizon = 7	ML predictor (<code>len(preds)=7</code> , confirmé par TTB=8 dans les logs)

11.2 Mécanisme de quantification $T_{send} \rightarrow T_{cycle}$

T_{send} réglé	T_{cycle} quantifié	Observé dans les logs
2 s	2 s en théorie	jitter réel 6–10 s (verrou anti-chevauchement, $T_{traitement} > 2s$)
4 s	4 s (marge nulle vs $T_{traitement} \approx 4, 2-4, 5s$)	jitter 4/6/8 s — instable
5 s	6 s ($\lceil 5/2 \rceil \times 2$)	stable, exactement 6 s , confirmé sur >60 cycles consécutifs

$T_{send}=5s$ est donc le seul réglage testé qui élimine le jitter — pas parce que 5 est une valeur magique, mais parce qu’il quantifie vers 6 s, qui laisse 1,5 s de marge au-dessus du temps de traitement mesuré (4,2-4,5 s).

11.3 Les prédictions ne sont pas géométriques

$\text{pred}[k] \approx L(d + (k+1)\delta)$ suppose que le modèle capture *parfaitement* la rampe. En réalité, le k-NN/ESN extrapole la **série temporelle de latence** sans connaître la position de la voiture. La formule donne la **cible de conception** (borne haute de proactivité) ; l’avance réelle dépend de la qualité du modèle. Si le modèle sous-estime (prédit $< \theta$ alors que le réel monte), la migration est tardive $\rightarrow$ réactive.

11.4 Mode réactif = filet de sécurité

La décision est **pilotée par la prédiction pour toutes les métriques** (primaire + secondaires) : `violation_detector.py` tranche sur les prédictions ML dès qu’elles sont disponibles. Le mode réactif n’apparaît que si l’API ML est indisponible (repli de sécurité). C’est cohérent avec la philosophie « proactif = décider sur le futur ». Ce comportement est renforcé par la **Règle A** (gate primaire, `decision.py`) et l’**hystérésis de migration** (5 %, voir §11.6), qui éliminent le thrashing indépendamment de la géométrie proactive (voir §1).

11.5 Optimisation Collector — cache de fond, gain non confirmé en pratique

Le `/collect` du hub interroge un **cache** rempli par un sondage de fond continu (`COLLECTOR_POLL_INTERVAL=1s`, `services/collector/collector.py`), découplé du cycle d’orchestration — `handle()` répond en 0,15-0,8 ms en **test isolé** (lecture pure du cache, sans appel réseau).

En conditions réelles cependant, sur un run live de 260+ cycles consécutifs, la durée mesurée de l’étape `collect` (chronométrée côté hub, appel HTTP complet hub $\rightarrow$ collector $\rightarrow$ hub) reste **systématiquement entre 700 et 1300 ms**, du premier cycle au dernier — le gain attendu du cache ne se matérialise pas dans le chemin critique du cycle. Cause probable, **non encore confirmée** : contention de l’évent loop `asyncio` entre la boucle de sondage de fond (appels réseau réels vers les 4 VMs) et le traitement de la requête HTTP entrante `/collect`, ou overhead de connexion HTTP non amorti. **Point ouvert** — à diagnostiquer avant d’envisager de réduire T_{send}/T_{cycle} sur la base de ce mécanisme.

11.6 Normalisation TOPSIS et marge d'hystérésis (5 %)

L'hystérésis anti-ping-pong (`_MIGRATION_MARGIN=0,05`, `decision.py`) exige que le meilleur candidat TOPSIS dépasse le score de la VM active d'au moins 5 % avant d'autoriser une migration — une zone morte qui absorbe le bruit de mesure/prédiction sans bloquer un vrai gain.

Ce mécanisme dépend cependant de la qualité de la normalisation TOPSIS. `TopsisSelector._minmax_normalize` (`topsis.py`) normalise chaque critère par $(valeur - min)/(max - min)$ **sur les seuls candidats du cycle courant**. Quand le filtre (`_filter_candidates`) ne retient que **2 candidats** aux valeurs quasi identiques (ex. latences prédites 96,454 ms vs 96,570 ms — écart réel de 0,12 %), cette formule polarise artificiellement l'écart en 0,000/1,000, comme si c'était l'écart maximal possible — le score de la VM active peut alors tomber exactement à **0,0**, ce qui neutralise la marge d'hystérésis ($0,0 \times 1,05 = 0,0$, n'importe quel challenger positif la franchit).

Garde-fou appliqué : un seuil de tolérance `_TIE_THRESHOLD=0,01` (1 %) dans `_minmax_normalize` — si l'écart relatif entre candidats sur un critère est en dessous de ce seuil, ils sont traités à égalité (norme 0,5 partout) plutôt que polarisés en 0/1. Validé par simulation sur des données réelles : deux candidats quasi identiques obtiennent alors un score égal (l'hystérésis bloque correctement la migration), sans affecter les cas à écart réel (testé sur un cycle à 4 candidats, écart 7,3 %, comportement inchangé).

11.7 Mode enhanced — SLOs CPU/RAM en ressource absolue (cœurs/Go)

En mode enhanced (intention utilisateur via LLM), les seuils CPU/RAM sont exprimés en **besoin absolu de ressource** (`operator: ">=", unit: "cores"/"GB"`) plutôt qu'en pourcentage de charge — ex. « *streaming vidéo fluide* » → `cpu_usage >= 2.0 cores`, `ram_usage >= 1.5 GB` — indépendamment de toute VM candidate précise. Ce choix tient compte de l'hétérogénéité du parc (edge : 4 cœurs, cloud : 8 cœurs), où un même pourcentage de charge ne représente pas la même marge réelle selon la capacité de la machine. Le prompt système du LLM inclut un **catalogue de profils de référence** (service léger, backend, streaming, ML) pour calibrer ce besoin à partir du type de service décrit, sans jamais exposer les capacités des VMs au modèle (séparation stricte besoin du service / infrastructure disponible).

Cette sémantique est appliquée de façon cohérente à deux niveaux (même conversion que celle utilisée pour le scoring TOPSIS, $total_cores \times (1 - usage\%/100)$) :

- `decision.py:_filter_candidates` — convertit la prédiction % en disponibilité absolue avant de comparer au seuil, quand `unit` est `cores/GB`.
- `violation_detector.py:_analyze/_severity` — même conversion, avec la direction de violation adaptée à l'opérateur : pour un plancher (`>=`), la violation survient **sous** le seuil (disponibilité insuffisante), à l'inverse d'un plafond (`<`) où elle survient au-dessus.

Confirmé en conditions réelles : une VM à ressources insuffisantes (edge, 1,8 cœur dispo pour un besoin de 2,0) est correctement écartée par le filtre et signalée en violation ; une VM largement suffisante (cloud, 6,6 cœurs dispo) ne l'est pas. Le tableau de bord (`observability/app.py`) affiche la disponibilité convertie (cœurs/Go) plutôt que le pourcentage brut, avec la même logique de direction.

12 Justification synthétique du choix

1. θ , B , A , D_{MIN} , D_{MAX} ne sont pas des choix arbitraires : $\theta=60$ ms est l'objectif métier initial (démon « QoS temps réel ») ; $B/A/D_{MIN}/D_{MAX}$ viennent du matériel déployé (4 scripts VM identiques, pas modifiables sans redéploiement).

2. $T_{send}=5s$ n'est pas choisi pour sa valeur faciale mais parce qu'il **quantifie vers 6 s**, seule cadence testée sans jitter — c'est une propriété du mécanisme de tick à 2 s du picar, pas un paramètre libre continu.
3. v est le seul levier vraiment libre restant, et sa valeur maximale utilisable est **contrainte par la géométrie** (Règle 5), pas par goût esthétique : à $T_{cycle}=6s$ fixé, $v=0,5$ respecte la règle avec marge, $v=1,0$ ne la respecte pas — preuve à l'appui dans les logs, pas seulement en théorie.
4. Si le produit souhaite néanmoins $v=1,0$ (démon plus dynamique), la seule compensation cohérente sans toucher à T_{cycle} est de **remonter θ** — ce n'est pas arbitraire non plus : la valeur 100 ms est calculée pour retrouver *exactement* la même marge de sécurité (D_{proac}) que le cas validé à $v=0,5$.

13 Procédure de changement de variable — garder le système stable

13.1 Carte d'impact (à consulter en premier)

Si changes...	tu Recalculer (dans l'ordre)	Règles à révéifier	Fichier à modifier	Redémarrage requis
θ (seuil)	$D_{slo} \rightarrow D_{proac}$	1, 2, 5	<code>shared/config.py</code>	<code>metrics_manager</code>
v	$\delta \rightarrow 7\delta \rightarrow D_{proac}$	3, 5	dashboard :8080 (slider)	aucun (runtime)
T_{send}	T_{cycle} (quantifié) $\rightarrow \delta \rightarrow 7\delta \rightarrow D_{proac} \rightarrow v_{max}$	toutes (1-6)	<code>infrastructure/picarx_sim.py</code>	sur le picar
B, A, D_{MIN}, D_{MAX}	$slope \rightarrow D_{slo} \rightarrow D_{proac} \rightarrow v_{max}$	toutes (1-6)	4 scripts VM + <code>picarx_sim.html</code>	4 VMs (SSH) + reload navigateur

Règle générale : **tout passe par slope et D_{slo} en premier**, puis par δ et D_{proac} — c'est la chaîne de dépendance du graphe §7. Ne jamais changer une valeur en aval (ex. v) sans révéifier ce qui est en amont (ex. si T_{cycle} a aussi changé entretemps).

13.2 Procédure A — Changer θ (seuil SLO)

1. Recalculer $D_{slo} = D_{MIN} + (\theta - B)/slope$ (slope inchangée, ne dépend pas de θ).
2. Recalculer $D_{proac} = D_{slo} - 7\delta$ (δ inchangé si v et T_{cycle} n'ont pas bougé).
3. Révéifier :
 - Règle 1 ($B < \theta/3$) — casse si θ devient trop petit
 - Règle 2 ($A > 2\theta$) — casse si θ devient trop grand
 - Règle 5 ($D_{proac} > D_{MIN}$) — c'est le levier utilisé pour compenser $v=1,0$ (θ : 60→100)
4. Si Règle 5 casse : soit remonter θ encore, soit revenir à la Procédure B ou C pour réduire δ à la place.
5. Modifier `shared/config.py` → `METRICS_REGISTRY["latency"]["default_threshold"]`.

6. **Redémarrer uniquement** `metrics_manager` (seul service qui lit `default_threshold` au chargement, dans `metrics_handler.py`: `select_dynamic_slos`) — `decision_intelligence` reçoit le seuil dans le payload à chaque cycle, pas besoin de redémarrage.
7. Revalider : faire tourner ≥ 10 cycles, vérifier qu'aucun `pred_breach=True` permanent n'apparaît (signe que Règle 5 tient en pratique, pas seulement sur le papier).

13.3 Procédure B — Changer v (vitesse voiture)

1. Recalculer $\delta = v \times T_{cycle}$ (T_{cycle} inchangé, supposé déjà stable — cf. Procédure C sinon).
2. Recalculer 7δ puis $D_{proac} = D_{slo} - 7\delta$ (D_{slo} inchangé si θ n'a pas bougé).
3. Revérifier :
 - Règle 3 ($v < v_{max} = (D_{slo} - D_{MIN})/T_{cycle}$)
 - Règle 5 ($D_{proac} > D_{MIN}$) — **c'est celle qui a cassé net à $v=1,0$ avec $\theta=60$**
4. Si Règle 5 casse : Procédure A (remonter θ) ou Procédure C (réduire T_{cycle}).
5. Modifier le slider **Vitesse** sur <http://<picar>:8080/> — paramètre runtime, **aucun service à redémarrer**.
6. Revalider : observer dans les logs `decision_intelligence` que `pred_breach` alterne entre `True/False` selon la position — pas figé sur `True` en continu.

13.4 Procédure C — Changer T_{send} (donc T_{cycle} , via quantification)

C'est la procédure la plus lourde car T_{cycle} **apparaît dans presque toutes les formules**.

1. Mesurer $T_{traitement}$ réel (durée **total** dans les logs hub, sur ≥ 10 cycles) — **ne jamais le deviner**.
2. Calculer $T_{cycle_quantifi} = \lceil T_{send}/2 \rceil \times 2$.
3. Vérifier Règle 6 ($T_{cycle_quantifi} > T_{traitement} + \text{marge de sécurité } 1,5 \text{ s}$) — sinon jitter garanti (observé à $T_{send}=4\text{s}$: cadence 4/6/8s erratique).
4. Une fois T_{cycle} stable trouvé, **recalculer toute la chaîne dans l'ordre** : $\delta = v \times T_{cycle} \rightarrow 7\delta \rightarrow D_{proac} = D_{slo} - 7\delta \rightarrow v_{max} = (D_{slo} - D_{MIN})/T_{cycle}$
5. Revérifier **les 5 règles géométriques** (elles dépendent toutes indirectement de T_{cycle} via δ ou v_{max}).
6. Modifier `infrastructure/picar_bridge.py` $\rightarrow$ `SEND_INTERVAL_S`, redémarrer `picar_bridge.py` sur le picar.
7. Revalider : observer les timestamps [`Cycle #N`] du hub sur ≥ 15 cycles — l'écart doit être **parfaitement constant**, pas de motif 4/6/8s.

13.5 Procédure D — Changer B, A, D_{MIN}, D_{MAX} (formule matérielle — rare)

1. **Modifier en cohérence les 5 endroits à la fois** : les 4 scripts VM (`*_ping_fixeCarac.py`) + `picarx_sim.html` (constante FML) — sinon le visuel et la latence réelle divergent (bug $D_{MAX}=70$ vs 80 déjà rencontré et corrigé).
2. Recalculer $\text{slope} = (A - B)/(D_{MAX} - D_{MIN})$ — **tout en aval en dépend**.

3. Recalculer D_{slo} , D_{proac} , v_{max} avec la nouvelle slope.
4. Revérifier les 5 règles géométriques.
5. Redéployer sur les 4 VMs (SSH), puis recharger le navigateur du picar (Ctrl+F5, pas de cache).
6. Revalider par un test de migration complet de bout en bout (pas juste un calcul).

13.6 Ce qui ne nécessite jamais de recalcul géométrique

- `MIGRATION_COOLDOWN_S`, la **marge d'hystérésis** (5 %, voir §11.6), et la **Règle A** (gate primaire) — mécanismes de *décision*, orthogonaux à cette géométrie de *déclenchement*. Les changer affecte le rythme des migrations, pas les formules D_{proac}/D_{slo} .
- `horizon` (=7) — fixé par l'API ML, pas un paramètre de démo.